BreakHis Breast Cancer Histopathology Classification

Future Reference Documentation

Project focus: Binary breast histopathology image classification using TensorFlow/Keras and transfer learning with MobileNetV2.

Classes: Benign (0) and Malignant (1).

Purpose of this document: A detailed technical record of the decisions, experiments, evaluation methodology, repository organization, and lessons discussed while developing the project. This is intended for future reference, not as the final public README.

Document date: September 2026

1. Project Overview

The project adapts a computer-vision challenge originally framed around pneumonia detection in X-ray images to a breast-cancer histopathology classification problem using the BreakHis dataset.

• Repository domain: Computer Vision

• Challenge type: Consolidation

• Challenge duration: 2 days

• Deployment strategy specified by the challenge: GitHub page

• Team format: Duo

• Core mission: design and evaluate a CNN for a practical classification task

• Original challenge mission: recognize pneumonia in X-rays

• Adapted mission: classify breast histopathology images as benign or malignant

Challenge requirements mapped to this project

Requirement	Project implementation
CNN trained on a dataset	MobileNetV2 transfer-learning classifier
Recognize unseen images	Held-out validation and test sets
Proper evaluation	Group-level split, confusion matrix, ROC, PR curve, classification metrics
Hyperparameter tuning	Multiple learning rates, fine-tuning depths, and class-weighted experiments
Visualizations	Training curves, confusion matrix, ROC, precision-recall, FP/FN image grids
Nice-to-have: feature maps	Not yet implemented
Nice-to-have: compare CNN structures	Not yet implemented
GitHub source code	Repository organized into source/data/model/results areas
README	To be prepared later

2. Dataset: BreakHis

The project uses the BreakHis breast histopathology dataset for binary classification. The working dataset contains 7,909 images.

Item	Value
Task	Binary classification
Class 0	Benign
Class 1	Malignant
Total images	7,909
Training images	5,303
Validation images	1,120
Test images	1,486
Image input size	224 × 224 × 3
Color	RGB

Important dataset property

BreakHis contains multiple images associated with the same underlying case/slide. Treating every image as independent during splitting can cause leakage: visually related images from the same case may appear in both training and evaluation data.

For this project, the split is performed at group level using a group identifier derived from the filename metadata: year + slide identifier. All images belonging to a group are assigned to exactly one split.

Leakage checks

• Train/validation group overlap: 0

• Train/test group overlap: 0

• Validation/test group overlap: 0

This is one of the most important methodological strengths of the project and should be emphasized in future documentation.

3. Dataset Split and Metadata

The split script creates an image-level CSV containing image paths, class labels, metadata, group identifiers, and the assigned split.

```
data/processed/breakhis_split.csv
```

The split is reproducible with random_state=42. The original approach creates a 70% training portion and a 30% holdout, then divides the holdout equally into validation and test.

Why group-level splitting matters

• Prevents the same case/slide from contributing images to both training and evaluation.

• Makes validation and test performance a more meaningful estimate of generalization.

• Reduces the risk of an artificially inflated score caused by near-duplicate or related images.

4. Preprocessing Pipeline

The preprocessing pipeline was implemented with TensorFlow rather than OpenCV. This is a deliberate adaptation of the original challenge instructions.

Pipeline

Stage	Implementation
Read image	TensorFlow file I/O
Color	RGB
Resize	224 × 224
Model preprocessing	MobileNetV2 preprocess_input
Model input range	Approximately [-1, 1]
Batch size	32
Prefetch	tf.data.AUTOTUNE
Augmentation	Training set only

The [-1, 1] point

MobileNetV2's preprocessing converts pixel values from the normal image representation to approximately [-1, 1]. Therefore, the model receives [-1, 1] tensors during training, validation, and test evaluation.

```
image = tf.keras.applications.mobilenet_v2.preprocess_input(image)
```

When an image is displayed for a human, the visualization code reverses that transformation approximately with (image + 1) / 2 and clips the result to [0, 1]. This is display-only. It does not change the data supplied to the model.

Augmentation

The current training augmentation consists of:

• Random horizontal and vertical flip

• Random rotation with factor=0.5

• Random zoom of 10%

Augmentation is applied only to training data. Validation and test images remain unaugmented.

Future consideration: factor=0.5 permits a large rotation range. It was intentionally preserved in the cleaned scripts because changing it would constitute a new experiment. If the project is experimentally extended, the effect of augmentation strength should be isolated and reported.

5. Model Architecture

The main architecture is ImageNet-pretrained MobileNetV2 used as a feature extractor, followed by a small binary classification head.

```
MobileNetV2 (ImageNet weights)
↓
GlobalAveragePooling2D
↓
Dropout(0.5)
↓
Dense(1, sigmoid)
```

Baseline strategy

• MobileNetV2 backbone initially frozen.

• Only the classification head is trained.

• Optimizer: Adam.

• Loss: Binary Cross-Entropy.

• Metrics: Binary Accuracy, Precision, Recall, AUC.

The model uses a sigmoid output because the task has two classes and malignant is represented as class 1.

6. Training Strategy

Frozen baseline

The baseline experiment freezes the MobileNetV2 feature extractor and trains the classification head. Multiple learning rates were explored.

Fine-tuning

Fine-tuning experiments progressively unfreeze a portion of the MobileNetV2 backbone. Batch Normalization layers are kept frozen during fine-tuning for stability.

Experiment	Configuration	Best validation AUC
1	Frozen, low LR (~1e-6)	≈ 0.567
2	Frozen, LR 1e-4	≈ 0.565

Experiment	Configuration	Best validation AUC
3	Frozen, LR 0.001	≈ 0.7003*
4	30-layer fine-tune, no class weights	≈ 0.6246
5	10-layer fine-tune, no class weights	0.7006
6	Class-weighted + 30-layer fine-tune	0.8112
7	Class-weighted + 15-layer fine-tune	0.8002
8	Class-weighted + 30-layer fine-tune rerun	0.8208

*Some earlier training-history notes contained an intermediate frozen-model AUC of approximately 0.6511, while the final experiment leaderboard used approximately 0.7003 for the frozen champion. Preserve the actual saved experiment logs/checkpoints if this historical distinction needs to be reconstructed.

Current validation champion

The current selected model is the class-weighted 30-layer fine-tuned MobileNetV2 checkpoint.

```
best_classweighted_30_finetune_layers.keras
```

Best validation AUC: **0.8208**, reached at approximately epoch 10 in the latest run.

7. Class Weighting

The class-weighted experiment uses balanced class weights calculated from the training split. The previously used values were approximately:

Class	Weight
Benign (0)	1.6606
Malignant (1)	0.7156

Class weighting changes the contribution of each class to the training loss. In this experiment it was combined with 30-layer fine-tuning and a learning rate of 1e-4.

Methodological caution: because class weighting and fine-tuning depth were changed together, the improvement from approximately 0.7006 to 0.8208 should not be presented as proof that class weighting alone caused the improvement. It is a combined experiment.

8. Model Selection and Test-Set Discipline

The project follows the important rule that the test set should be used only after model selection.

• Training data is used to fit model parameters.

• Validation data is used to compare experiments and select the best configuration.

• The test set is held out for final evaluation.

• Do not tune the architecture, learning rate, class weights, or classification threshold based on final test results.

This distinction is especially important for a medical-imaging project because repeatedly inspecting test performance and adapting the model to it can turn the test set into an indirect training resource.

9. Final Test Evaluation

The final selected class-weighted 30-layer fine-tuned model was evaluated on 1,486 held-out test images.

Metric	Test result
Accuracy	0.6743
Precision — malignant	0.7860
Recall — malignant	0.6016
F1-score — malignant	0.6816
ROC AUC	0.7622

Confusion matrix

	Predicted Benign	Predicted Malignant
Actual Benign	484 (TN)	141 (FP)
Actual Malignant	343 (FN)	518 (TP)

The AUC of 0.7622 was computed from the complete collected test probabilities using sklearn's roc_auc_score. The Keras batch-aggregated AUC reported approximately 0.7606; the project should use one clearly defined calculation consistently in the final report, preferably the probability-array calculation.

Validation vs test

Validation AUC was 0.8208 and test AUC was 0.7622, a difference of approximately 0.0586. This gap should be discussed as evidence that validation performance did not fully transfer to the held-out test set.

Medical-context interpretation

The malignant recall of 0.6016 means that 343 of the 861 malignant test images were classified as benign. These false negatives are particularly important in a diagnostic context because a missed malignant case can be more consequential than a false positive.

The project should therefore avoid presenting accuracy alone as the key measure. Recall/sensitivity for malignant cases, precision, F1, ROC AUC, and the confusion matrix provide a more informative picture.

10. ROC and Precision-Recall Analysis

Two threshold-independent/threshold-sensitive visual analyses were added to the final evaluation.

ROC curve

The ROC curve plots true-positive rate against false-positive rate across classification thresholds. The final test AUC is approximately 0.7622.

```
results/evaluation/final_test_roc_curve.png
```

Precision-recall curve

The precision-recall curve is particularly useful when the positive class is clinically important and class balance is imperfect. Here, malignant is treated as the positive class.

```
results/evaluation/final_test_precision_recall_curve.png
```

These plots are descriptive evaluation artifacts. They should not be used to tune the final test threshold after seeing test performance.

11. False-Positive and False-Negative Analysis

Definitions

• **False positive (FP):** actual benign, predicted malignant.

• **False negative (FN):** actual malignant, predicted benign.

The error-analysis script uses the same held-out test metadata and selected final model. It identifies incorrectly classified test images and creates visual grids for qualitative inspection.

Current error counts

• False positives: 141

• False negatives: 343

The script can select the most confident errors for visual inspection: high malignant probability among false positives and low malignant probability among false negatives.

Important preprocessing distinction

The model still receives MobileNetV2-preprocessed values in approximately [-1, 1]. The error-analysis display loads the original image and scales it to [0, 1] only because matplotlib expects a conventional display range. This is not a model input transformation.

Purpose of FP/FN analysis

• Look for systematic visual patterns in errors.

• Check whether errors are concentrated at certain magnifications or tumor subtypes.

• Identify potentially ambiguous examples.

• Generate qualitative evidence for the project discussion.

• Do not use the held-out test errors to tune the model.

12. Training and Evaluation Visualizations

The project now includes the core visualizations expected by the challenge.

Visualization	Purpose
Loss curve	Check optimization and overfitting behavior
Accuracy curve	Track classification accuracy
Precision curve	Track positive-prediction quality
Recall curve	Track malignant detection sensitivity
AUC curve	Track ranking performance during training
Confusion matrix	Show TP/TN/FP/FN counts
ROC curve	Show sensitivity/specificity trade-off
Precision-recall curve	Show precision/recall trade-off
FP examples	Qualitative analysis of benign images called malignant
FN examples	Qualitative analysis of malignant images called benign

13. Professional Repository Structure

```
CV_Histopath_Cancer_Diagnosis/
■■■ data/
■ ■■■ raw/ # Local BreakHis dataset; ignored by Git
■ ■■■ processed/ # Generated split metadata
■■■ models/
■ ■■■ experiments/ # Experiment checkpoints
```

```
■ ■■■ final_model.keras # Optional selected final checkpoint
■■■ results/
■ ■■■ training_curves/ # Training/validation plots
■ ■■■ evaluation/ # Final test evaluation plots
■ ■■■ error_analysis/ # FP/FN visual analysis
■ ■■■ preprocessing/ # Preprocessing previews
■■■ src/
■ ■■■ data_analysis.py
■ ■■■ train_val_test.py
■ ■■■ preprocessing.py
■ ■■■ train_model.py
■ ■■■ finetune_model.py
■ ■■■ class_weight_model.py
■ ■■■ test_evaluation.py
■ ■■■ error_analysis.py
■■■ README.md
■■■ requirements.txt
■■■ .gitignore
■■■ LICENSE
```

Role of each Python script

File	Role
data_analysis.py	Explore dataset structure, classes, magnifications, and groups.
train_val_test.py	Create and verify the leakage-safe group-level split.
preprocessing.py	Create TensorFlow datasets, normalization, and training augmentation.
train_model.py	Train the frozen MobileNetV2 baseline and save training curves.
finetune_model.py	Fine-tune the top 10 MobileNetV2 layers.
class_weight_model.py	Run the class-weighted 30-layer fine-tuning experiment.
test_evaluation.py	Perform final held-out test evaluation and save metrics plots.
error_analysis.py	Inspect and visualize false positives and false negatives.

14. Code Quality and Documentation Conventions

The cleaned scripts were reorganized to follow a consistent professional Python style.

• Every script has a module-level docstring explaining its purpose.

• Functions have docstrings describing their responsibilities and important inputs/outputs.

• Constants are grouped near the top of the script.

• Repository paths are derived from the script location instead of relying on the current working directory.

• Execution is protected by an if __name__ == '__main__' entry point.

• Comments explain important methodological decisions rather than narrating obvious syntax.

• Long procedures are divided into focused functions.

• Error messages explicitly identify missing files or invalid data.

This organization is intended to make the project easier to understand during review, debugging, GitHub publication, and future extension.

15. .gitignore

The original .gitignore contained repeated entries for Python caches, virtual environments, macOS files, and the BreakHis dataset. These redundancies were removed.

Important ignored content

```
__pycache__/
*.py[cod]

.venv/
venv/
env/

.env
.envrc

.pytest_cache/
.coverage
.mypy_cache/
.ruff_cache/

.DS_Store

.vscode/
.idea/

data/raw/
breakhis/

*.h5
```

The raw BreakHis dataset is ignored because it is large and should not be committed to the repository.

The cleaned .gitignore does not automatically ignore .keras files. This leaves the project free to decide whether selected Keras checkpoints should be versioned, stored externally, or excluded later.

16. Challenge Requirement Checklist

Requirement	Status	Notes
Train a CNN	Completed	MobileNetV2 transfer learning
Classify unseen images	Completed	Held-out test set
Train/validation/test split	Completed	Group-level split
Prevent leakage	Completed	Group overlap checks = 0
Hyperparameter experimentation	Completed	Multiple configurations tested
Confusion matrix	Completed	Final test set
ROC curve	Completed	Final test set
Precision-recall curve	Completed	Final test set
Training curves	Completed	Loss, accuracy, precision, recall, AUC
Discuss relevant hospital metrics	Completed	Recall/FN emphasized
Feature maps	Not yet	Nice-to-have
Compare CNN structures	Not yet	Nice-to-have
GitHub source code	Organization prepared	Final publication still needed
README	Not yet	Intentionally postponed

17. Important Methodological Lessons

• **Never split histopathology images blindly at image level** when multiple images belong to the same case/slide.

• **Validation performance is for model selection.** Test performance is for final reporting.

• **Accuracy is not enough** for a medical classification task.

• **False negatives deserve special attention** because malignant cases classified as benign are clinically concerning.

• **Class weighting and fine-tuning depth were combined** in the strongest experiment, so their individual causal effects cannot be separated from this experiment alone.

• **Visualization preprocessing is not model preprocessing.** Model inputs remain approximately [-1, 1]; display images can be converted to [0, 1].

• **Clean repository structure matters** for reproducibility and professional presentation.

• **Do not silently change experimental settings during code cleanup.** A cleaned script should represent the experiment that was actually run.

18. Recommended Next Steps

Priority	Next action
1	Freeze the current final test results and do not tune against the test set.
2	Keep the group-level split and document it prominently.
3	Inspect FP/FN grids and record qualitative observations.
4	Decide whether the 0.5 classification threshold is only a default or whether a threshold-selection experiment should be performed using validation data.
5	Optionally add feature-map visualization as the challenge nice-to-have.
6	Optionally compare MobileNetV2 with another CNN architecture.
7	Decide how model checkpoints will be handled in GitHub.
8	Create the final README only after the project structure and experiments are frozen.

If threshold optimization is pursued

Any threshold optimization should be performed using the validation set, not the final test set. Once a threshold is selected and frozen, it can be applied once to the held-out test set for final reporting.

19. Reference: Current Key Numbers

Item	Value
Dataset size	7,909 images
Train / validation / test	5,303 / 1,120 / 1,486
Input	224 × 224 RGB
Model input range	Approximately [-1, 1]
Batch size	32
Final model	MobileNetV2 + GAP + Dropout(0.5) + Dense(1, sigmoid)
Final experiment	Class weighting + top 30-layer fine-tuning
Learning rate	1e-4
Best validation AUC	0.8208
Final test accuracy	0.6743
Final test malignant precision	0.7860
Final test malignant recall	0.6016
Final test malignant F1	0.6816
Final test AUC	0.7622

Item	Value
TN / FP / FN / TP	484 / 141 / 343 / 518

20. Final Reference Notes

This document captures the project state and decisions discussed across the working sessions. It is intentionally more detailed and technical than the future public README.

When updating the project, distinguish three things clearly: (1) the experiments that were actually run, (2) proposed future experiments, and (3) changes made only for code organization. Code cleanup should not be described as a new modeling experiment.

The strongest current result is the class-weighted 30-layer fine-tuned MobileNetV2 model with validation AUC 0.8208 and final held-out test AUC 0.7622. The test set remains the final evaluation set and should not be used for further model tuning.